

Serving: turning Gold data into *useful* data

By now, we have learned to move data from Bronze (raw) to Silver (cleaned) and finally to Gold (aggregated KPIs). However, data sitting in a storage bucket isn't "useful" yet.

In this lab, we will implement the Serving Layer. We will treat our Gold table as a product and expose it through three different interfaces tailored to different consumers:

- SQL Views: for BI analysts and SQL power users
- REST API: for application developers and real-time microservices
- MCP Server (Model Context Protocol): for AI Agents (like Claude) to interact with your data

1. Initialize the Gold Dataset

First, let's ensure we have a stable Gold table in our Lakehouse (Trino + Iceberg) to work with.

Execute the following SQL to create the gold.customer_daily_kpis table and populate it with sample data.

```
CREATE SCHEMA IF NOT EXISTS iceberg.gold;

CREATE TABLE IF NOT EXISTS iceberg.gold.customer_daily_kpis(
  customer_id VARCHAR,
  event_date DATE,
  total_revenue DOUBLE,
  order_count INTEGER,
  avg_order_value DOUBLE,
  is_active_user BOOLEAN
) WITH (
  format = 'PARQUET',
  partitioning = ARRAY['event_date'] );
-- Insert mock data for testing
INSERT INTO iceberg.gold.customer_daily_kpis VALUES
('C_001', DATE '2026-03-30', 250.00, 2, 125.00, true),
('C_002', DATE '2026-03-30', 10.00, 1, 10.00, true),
('C_001', DATE '2026-03-31', 50.00, 1, 50.00, true),
('C_003', DATE '2026-03-31', 1200.00, 5, 240.00, true);
```

2. Interface 1 – The Semantic View (SQL)

Raw Gold tables can still be too "wide" or contain technical flags. We use Views to create a semantic layer. Create a view specifically for the Marketing Team. They only care about "VIP" customers (those who have spent more than \$500 in total).

```
CREATE OR REPLACE VIEW iceberg.gold.v_marketing_vip_customers AS
```

```
SELECT
  customer_id,
  sum(total_revenue) as lifetime_spend,
  sum(order_count) as total_orders
FROM iceberg.gold.customer_daily_kpis
GROUP BY customer_id
HAVING sum(total_revenue) > 500;
```

3. Interface 2 – Data-as-a-Service (REST API)

Web applications shouldn't query your Lakehouse directly via SQL (for security and connection pooling reasons). We'll wrap the data in a FastAPI endpoint.

Implement a simple API in Python. Ensure you have `fastapi` `uvicorn` `trino` installed.

```
from fastapi import FastAPI
import trino

app = FastAPI(title="ShopNow Data API")

def get_trino_client():
    return trino.dbapi.connect(
        host='host.docker.internal',      port=8080,      user='tead',
        catalog='iceberg', schema='gold'
    )

@app.get("/metrics/{customer_id}")
def get_customer_summary(customer_id: str):
    conn = get_trino_client()
    cur = conn.cursor()

    cur.execute(f"""
        SELECT sum(total_revenue), sum(order_count)
        FROM customer_daily_kpis
        WHERE customer_id = '{customer_id}'
    """)
    res = cur.fetchone()
    return {
        "customer_id": customer_id,
        "total_revenue": res[0] if res[0] else 0,
        "total_orders": res[1] if res[1] else 0
    }
```

If you save this script in a `main.py` file, you can start the API through `uvicorn main:app --reload`.

Next, you can try the API through your browser or apps like Postman, by calling the endpoint `http://127.0.0.1:8000/metrics/C_001`.

FastAPI also automatically generates documentation. You can visit <http://127.0.0.1:8000/docs> to consult the documentation and test the API without writing a single line of frontend code.

3.1 Interface 2 – Data-as-a-Service (REST API) with documentation

Let us now improve the documentation of the API, so that users can navigate it as part of the semantic layer. This allows potential users to clearly get to know everything that is available, without having to contact us to find out.

First, let's improve the general documentation and defining the "contract" of the data product right at the entry point. Change the initialization of the API to:

```
app = FastAPI(
    title="ShopNow Gold Data API",
    description="""
        This API provides access to the Gold Layer of the ShopNow
        Lakehouse.

        ### Features:
        * Customer Metrics: Daily aggregated KPIs.
        * Latency: Data updated every 24h (Batch).
        * Owner: Data Engineering Team (MEI-TEAD).
        """,
    version="1.0.0",
)
```

FastAPI also automatically picks up Python docstrings and uses them as the description for the endpoint in the Swagger UI.

So, now, let's add docstrings to our only method:

```
@app.get("/metrics/{customer_id}", tags=["Customer Analytics"])
def get_customer_summary(customer_id: str):
    """
    Fetch daily KPIs for a specific customer.

    - customer_id: The unique identifier (e.g., C_001)
    - Returns: Total revenue and order count aggregated from the
    Iceberg Gold table.
    """
    # ... logic here ...
```

Another important part is to define the Schema. By using a Pydantic model, we can document exactly what each field means and even provide examples. Change the script to the following:

```

from pydantic import BaseModel, Field

class CustomerMetrics(BaseModel):
    customer_id: str = Field(..., example="C_001", description="Unique identifier for the customer")
    total_revenue: float = Field(..., example=150.50, description="Sum of all purchases in EUR")
    total_orders: int = Field(..., example=3, description="Count of successful checkout events")

@app.get("/metrics/{customer_id}", tags=["Customer Analytics"], response_model=CustomerMetrics)
def get_customer_summary(customer_id: str):
    # FastAPI will now show the schema and example in the docs!
    ...

```

When there are parameters (e.g. filtering by a date), we can also document them using `Query`:

```

from fastapi import Query

@app.get("/metrics/{customer_id}")
def get_customer_summary(
    customer_id: str,
    days_back: int = Query(7, description="Number of days to look back for aggregations", ge=1, le=30)
):
    ...

```

4. Interface 3 – The Agent / LLM interface (MCP)

The Model Context Protocol (MCP) is an open standard that lets LLMs (like Claude) safely use tools and data. Let's start by creating an `mcp_server.py`. This will allow an AI to "know" about your data without you having to copy-paste it.

```

from mcp.server.fastmcp import FastMCP
import trino

mcp = FastMCP("ShopNow-Analytics")

@mcp.tool()
def get_high_level_stats() -> str:
    """Returns the total revenue and active customer count from the Gold layer."""
    conn = trino.dbapi.connect(host='host.docker.internal', port=8080, user='tead', catalog='iceberg', schema='gold')
    cur = conn.cursor()
    cur.execute("SELECT sum(total_revenue), count(distinct customer_id) FROM customer_daily_kpis")
    rev, count = cur.fetchone()

```

```
    return f"The store has generated ${rev:.2f} in revenue from {count}
unique customers."

if __name__ == "__main__":
    mcp.run()
```

The easiest way to test the MCP server is to open the Claude desktop app. You also need to tell Claude where your server is. You do this by editing a JSON file:

- macOS: ~/Library/Application Support/Claude/claude_desktop_config.json
- Windows: %APPDATA%\Claude\claude_desktop_config.json

Then, add your server to the config:

```
{
  "mcpServers": {
    "shopnow-analytics": {
      "command": "python",
      "args": [
        "/absolute/path/to/your/mcp_server.py"
      ],
      "env": {
        "TRINO_HOST": "host.docker.internal"
      }
    }
  }
}
```

After saving this file, you must restart Claude Desktop (quit and reopen). You can now ask Claude questions that relate to your data product, and it will answer based on data retrieved from the Gold product.

Once configured, a small "hammer" icon (tools icon) will appear in the Claude chat box. You can then simply type: "What is the total revenue for customer C_001?". Claude will recognize the request, realize it has a tool named `get_customer_metrics`, call your Python script, get the data from Trino, and summarize the answer for the user.

If you don't want to open Claude every time you change a line of code, you can also use the MCP Inspector. It is a specialized CLI tool that simulates an LLM connection.

To run it, in your terminal, inside your project folder, run:

```
npx @modelcontextprotocol/inspector python mcp_server.py
```

It provides a web UI (usually at <http://localhost:3000>) that lists all your tools instantly.

You can click "Run" on a tool to see if the SQL query to Trino is actually working without waiting for an LLM to "think."

4.1 Interface 3 – The Agent / LLM interface (MCP) with documentation

When you use a library like FastMCP, the SDK uses Python's introspection to read the `__doc__` attribute of your function. It then packages that string into the description field of the JSON object sent to the LLM.

This is important as an LLM doesn't see your code: it only sees the Tool Definition. If your docstring is poor, the agent might:

- Hallucinate: it might guess what the parameters do
- Misuse: it might call a "Heavy Query" tool for a simple question
- Fail to trigger: it might not realize your tool can answer the user's question

We must thus get used to treat docstrings as Metadata. The example below, is a bad / lazy way of doing it:

```
@mcp.tool()
def get_customer_revenue(customer_id: str):
    """Gets revenue for a customer."""
    ...
```

The correct alternative would be something like:

```
@mcp.tool()
def get_customer_revenue(customer_id: str):
    """
    Retrieves the total accumulated revenue for a specific customer from
    the Gold Layer.

    Use this tool when:
    - You need the lifetime value (LTV) of a specific user.
    - The user asks 'How much did [ID] spend?'.

    Notes:
    - Revenue is returned in EUR (€).
    - Data grain is daily, but this tool returns the SUM across all
    available dates.
    - If a customer_id is not found, it returns 0.0.
    """
    ...
```

If you completed the tutorial, you now know how to provide different types of access to your data, to different types of users. Next, you should document all these forms of access in a single document, such as a markdown file, describing the underlying product, all the different access types available, and who they are meant to.